

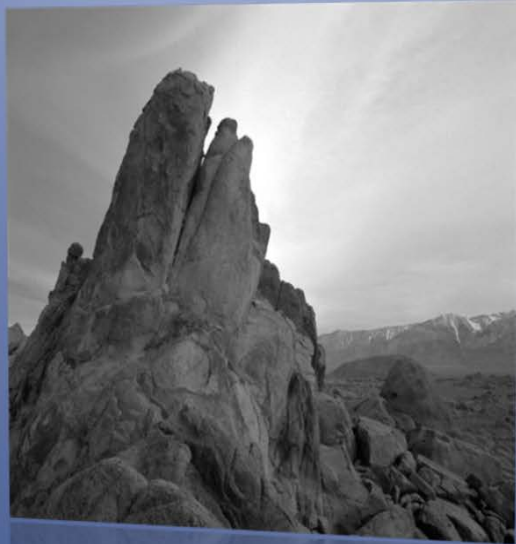
Cấu Trúc Dữ Liệu & Giải Thuật

Khoa CNTT

Email: ptbao@sgu.edu.vn

Nội dung chương trình

1. Tìm kiếm và sắp xếp (Search & Sort)
2. CTDL Danh sách liên kết (Linked List)
3. CTDL Cây (Tree)
4. Băm
5. Nội dung thêm



CẤU TRÚC DỮ LIỆU VÀ THUẬT GIẢI

PHẠM THẾ BẢO

Giáo trình

Tài liệu tham khảo

1. Data Structures and Algorithm Analysis in C, Mark Allen Weiss, Addison Wesley Longman Inc. – 1997.
2. Nhập môn Cấu Trúc Dữ Liệu và Thuật Toán, TS. Dương Anh Đức, ĐHQG Tp.HCM
3. Data Structures and Program Design in C++, Robert L. Kruse and Alexander J. Ryba, Prentice – Hall International Inc. – 1999.
4. Data Structures and C Programs, Christopher J. Van Wyk, Addison Wesley – 1992.
5. Cấu Trúc Dữ Liệu + Thuật Giải = Chương Trình, người dịch Nguyễn Quốc Cường, nhà xuất bản Đại học và Giáo dục chuyên nghiệp.
6. Internet

Tìm kiếm và sắp xếp

1. 02 thuật toán tìm kiếm
2. 11 thuật toán sắp xếp

Cấu trúc dữ liệu DSLK

- DSLK đơn
- DSLK đôi, đa
- Ngăn xếp (Stack)
- Hàng đợi (Queue)
- DSLK có thứ tự
- DSLK vòng

Cấu trúc dữ liệu cây

- Cây tổng quát
- Cây nhị phân
- Cây nhị phân tìm kiếm

Băm (Hashing)

- Hàm băm
- Bảng băm
- Giải quyết đụng độ

Nội dung thêm

- Cây AVL (AVL Tree)
- Cây đỏ đen (Red-Black Tree)
- Tìm kiếm chuỗi (Pattern Matching)

Tổng quan

- Khái niệm thuật giải
- Phân tích thuật giải
- Trừu tượng hoá dữ liệu

Khái niệm thuật giải

- Thuật giải là gì ?
- Tính chất của thuật giải:
 1. Dữ liệu
 2. Tính xác định
 3. Tính dừng
 4. Tính đúng đắn
 5. Tính khả thi

Khái niệm thuật giải (tt)

- Tất cả các bài toán đều giải được trên máy tính ?
- Có bao nhiêu thuật giải ?
- Mục tiêu của thuật giải là gì ?
- Phải chọn thuật giải nào ?

Phân tích thuật giải

- Thời gian thực hiện (Running Time)
- Mã giả (Pseudo-Code)
- Phân tích thuật giải

Thời gian thực hiện (Running Time / Time Complexity)

- Thời gian thực hiện phụ thuộc gì ?
- Vậy làm sao đánh giá thuật giải ?
- Làm thế nào xác định thời gian thực hiện ?

Hướng tiếp cận thực nghiệm

- Các bước thực hiện:
 1. Viết chương trình cài đặt
 2. Thực thi chương trình với nhiều bộ dữ liệu
 3. Đo và thống kê thời gian
 4. Xấp xỉ biểu đồ
- Hạn chế:
 1. Cần phải cài đặt CT và đo thời gian
 2. Bộ dữ liệu không thể đặc trưng hết
 3. Khó so sánh 02 thuật giải

Mã giả

- Mã giả là gì ?
- Đặc tính cơ bản:
 1. Mô tả thuật giải có cấu trúc hơn NNTN
 2. Kém hình thức hơn NNLT
 3. Công cụ chủ yếu mô tả thuật giải
 4. Tàng ẩn dưới khái niệm thiết kế CT

Mã giả (tt)

- Một số quy ước:
 1. Các biểu thức toán học
 2. Lệnh gán: “←”
 3. So sánh: “=“
 4. Khai báo hàm (thuật giải)

Thuật giải <tên TG> (<tham số>)

Input: <dữ liệu vào>

Output: <dữ liệu ra>

<Các câu lệnh>

End <tên TG>

Mã giả (tt)

5. Các cấu trúc:

- Cấu trúc chọn:

If ... then ... [else ...]

- Vòng lặp:

While ... do

Do ... while (...)

For ... do ...

6. Một số câu lệnh khác:

- Trả giá trị về: Return [*giá trị*]

- Lời gọi hàm: <Tên>(tham số)

Phân tích thuật giải

- Khái niệm độ phức tạp của thuật toán
- Các thao tác cơ sở
- Khảo sát mã giả
- Ví dụ

Thuật giải MaxOfArray(A,n)

Input: Mảng A có n phần tử

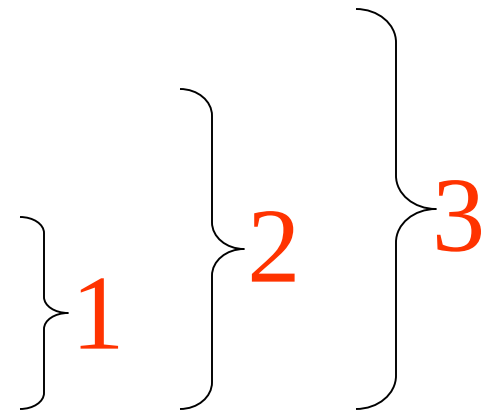
Output: Phần tử lớn nhất của A

currentMax $\leftarrow$ A[0]

for (i $\leftarrow$ 1 to n-1) do

 if(currentMax < A[i])

 currentMax $\leftarrow$ A[i]



return currentMax

End MaxOfArray

TT	Thao tác	Tốt nhất	Xấu nhất	TB
1	So sánh	$n-1$	$n-1$	$n-1$
2	Gán	1	n	$n/2$
3	Return	1	1	1

Xấp xỉ tiệm cận

- Mục đích

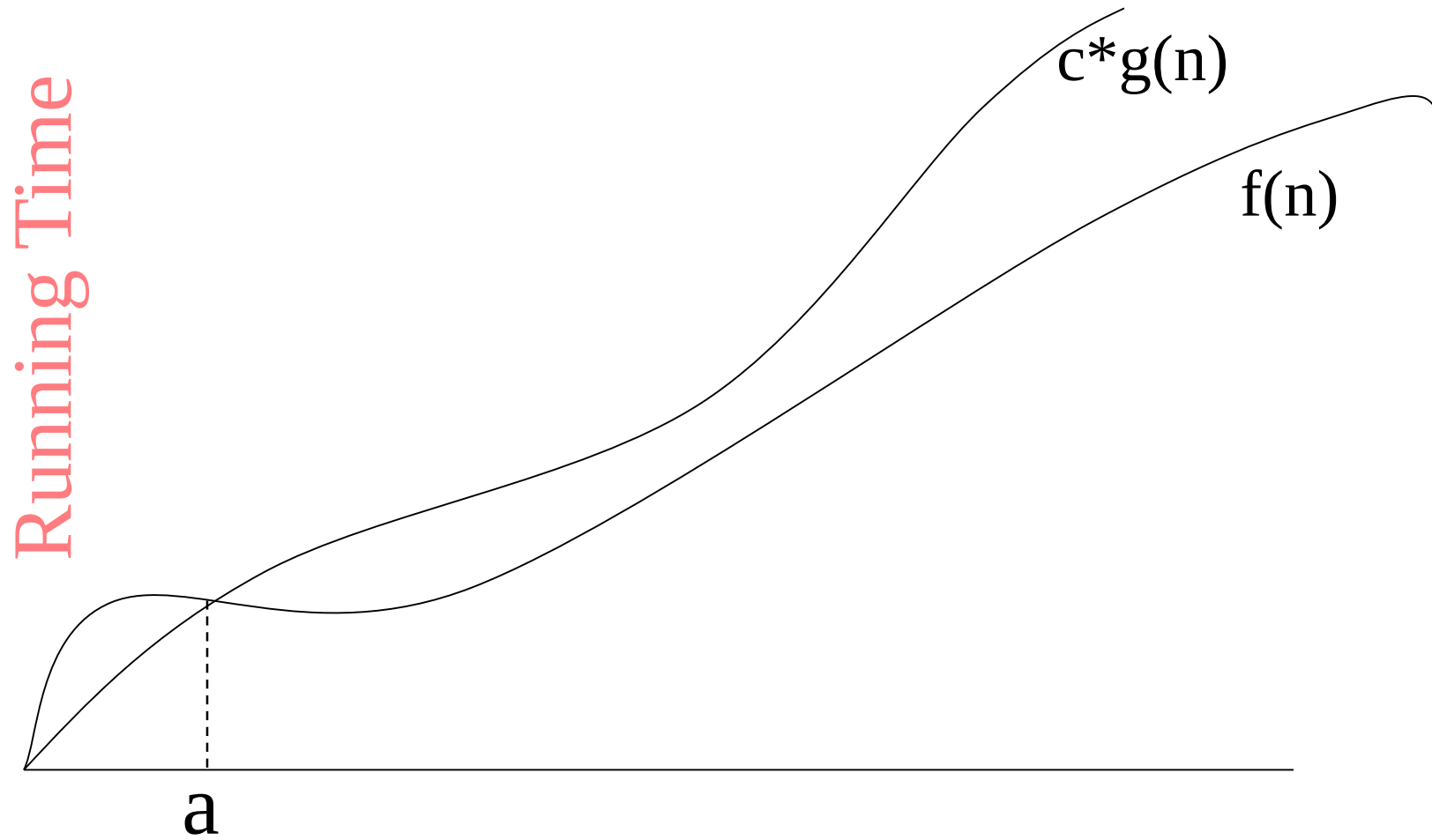
- o Ví dụ:

- $1,000,001 \approx 1,000,000$

- $3n^2 \approx n^2$

- Khái niệm “O”:

- o Cho $f(n)$ và $g(n)$ ta nói $f(n) \in O(g(n))$ khi và chỉ khi $f(n) \leq c \cdot g(n)$, $\forall n \geq a$ với c và a là hằng số, và $f(n)$ và $g(n)$ là các hàm trên miền số tự nhiên.



Xấp xỉ tiệm cận (tt)

- o Nếu $7n^2 - 3 \in O(n^5)$ nhưng ta chỉ quan tâm đến xấp xỉ bậc thấp nhất $\Rightarrow 7n^2 - 3 \in O(n^2)$
- o Quy tắc xác định $O(?)$: xét thành phần có bậc cao nhất của f

Ví dụ:

$$12n - 2 \in O(n)$$

$$3n^2 \log(n) - 12n^2 + 19 \in O(n^2 \log(n))$$

$$3e^n - 10000n^2 + 2 \in O(e^n)$$

Phân tích thời gian theo hướng tiệm cận

- Sử dụng ký hiệu “O” để biểu diễn khối lượng các thao tác cơ sở.

Ví dụ: ta có thể nói hàm MaxOfArray chạy với thời gian $O(n)$

- Khi so sánh các thuật toán ta có thể nói:

Thuật toán chạy với thời gian $O(n)$ tốt hơn thuật toán $O(n^2)$

Tương tự $O(\log(n))$ tốt hơn $O(n)$

Phân tích thời gian theo hướng tiệm cận (tt)

- Có thể hệ số ứng với thành phần có bậc cao nhất có thể rất lớn nên đôi khi, với tập dữ liệu Input xác định thuật toán $O(n)$ sẽ kém hiệu quả hơn thuật toán $O(n^2)$ hoặc ngay cả $O(2^n)$

Ví dụ: $10,000,000n$ là $O(n)$ nhưng kém hiệu quả hơn $O(n^2)$ khi $n \ll 10,000,000$

Phương pháp thực hiện

- Lý thuyết hàm sinh
- Khảo sát thực tế

Ví dụ: $\log(n) < \text{thực tế} \lll n$

- Xét ví dụ đoạn chương trình:

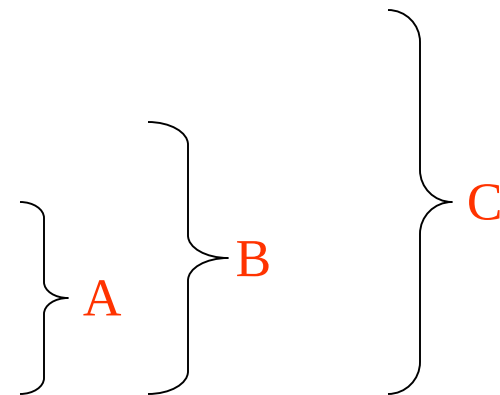
```
for(i ← 1 to n) do
```

```
    if(i < 10) then
```

```
        for(j ← 1 to log(n)) do
```

```
            print j
```

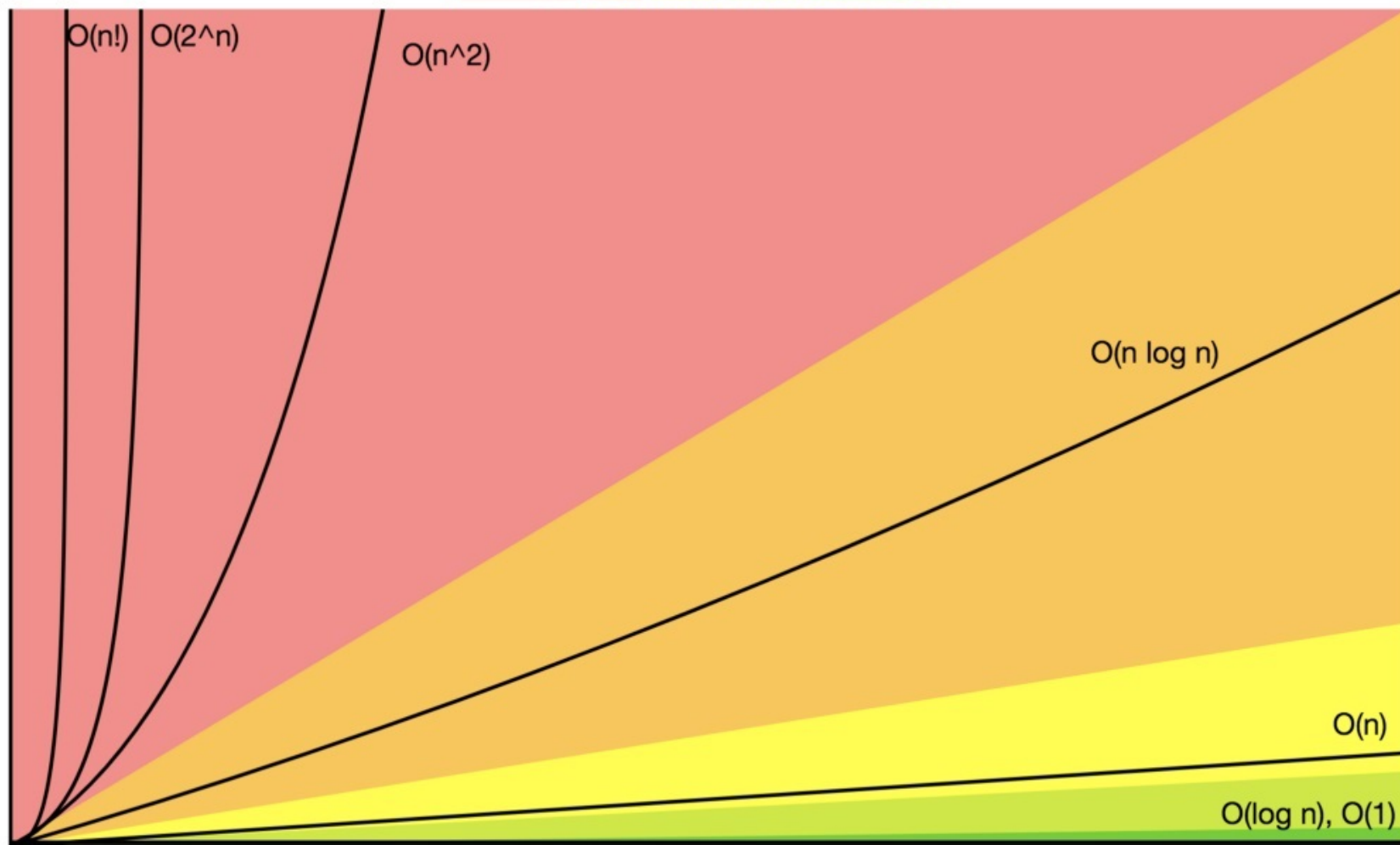
```
    else print i
```



Phân tích ví dụ

- $T(A) \in O(\log(n))$
 - $T(B) \in O(1) + \text{Max}\{O(\log(n)), O(1)\}$
 $= O(1) + O(\log(n))$
 $= O(1 + \log(n))$
 $= O(\log(n))$
- $T(C) \in n * O(\log(n))$
 $= O(n * \log(n))$

Rất tệ Tệ Khá Tốt Rất tốt



Số phần tử

Trừu tượng hoá dữ liệu

- Đời sống của một chương trình:
 1. Mô tả bài toán (Task specification)
 2. Thiết kế (Design)
 3. Kiểm tra (Verification)
 4. Cài đặt (Implementation, Coding)
 5. Thử nghiệm, bảo trì, nâng cấp, hoàn thiện và sử dụng (Testing Maintenance, Evolution, Refinement and Use)
 6. Lỗi thời và biến mất

Trừu tượng hoá dữ liệu (tt)

- Thế nào là một chương trình tốt ?
- Các tính chất của chương trình tốt:
 1. Đúng đắn (Correct)
 2. Hiệu quả (Efficient)
 3. Chi phí thấp (Cheap)
 4. Dễ hiểu (Understandable)
 5. Dễ bảo trì, nâng cấp (Maintainable)
 6. Mạnh mẽ (Robust)
 7. Đơn thể (Modular)

Trừu tượng hoá dữ liệu (tt)

- Kỹ thuật *Lập Trình Đơn Thể* là gì ?
- Dùng phương pháp thiết kế nào để xác định các đơn thể ?
 1. Top-Down
 2. Hoặc Bottom-Up
- LTĐT & trừu tượng hoá để làm gì ?
- Cơ sở chính của trừu tượng hoá:
 - ✓ Quan tâm tổng thể
 - ✓ Không quan tâm chi tiết
 - ✓ Tương tự khái niệm “*Hộp đen*”

Trừu tượng hoá dữ liệu (tt)

- Phân loại trừu tượng hoá:
 1. Trừu tượng hoá chức năng (Functional Abstraction), phân tách giao tiếp của chức năng (hàm) với cài đặt cụ thể.
 2. Trừu tượng hoá dữ liệu (Data Abstraction), phân biệt thuộc tính dữ liệu với cài đặt cụ thể.

Trừu tượng hoá dữ liệu (tt)

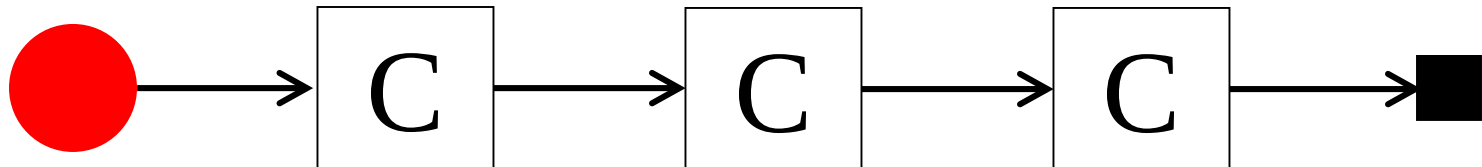
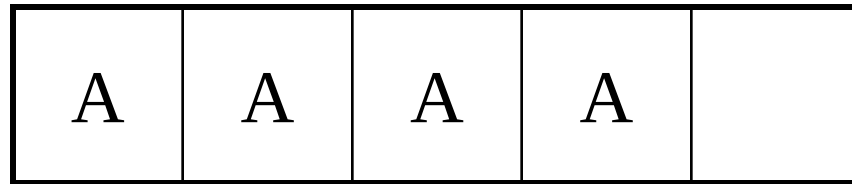
- Một kiểu dữ liệu bao gồm những gì ?
 1. Một tập hợp các giá trị
 2. Một tập hợp các phép toán (thao tác) trên các giá trị này
- Kiểu dữ liệu trừu tượng (*Abstract Data Type* - **ADT**) sẽ do người dùng định nghĩa
- Cần chú ý khi cài đặt ADT:
 1. Chọn biểu diễn thích hợp của dữ liệu thông qua các kiểu dữ liệu đã biết.
 2. Cài đặt các thao tác trên dữ liệu

Trình tượng hoá dữ liệu (tt)

- Kiểu dữ liệu cơ sở
Ví dụ : char, int, boolean, ...
được định nghĩa sẵn trong NNLT
- Kiểu dữ liệu (KDL) có cấu trúc
- Phân loại KDL:
 1. Tuyến tính
 2. Phi tuyến

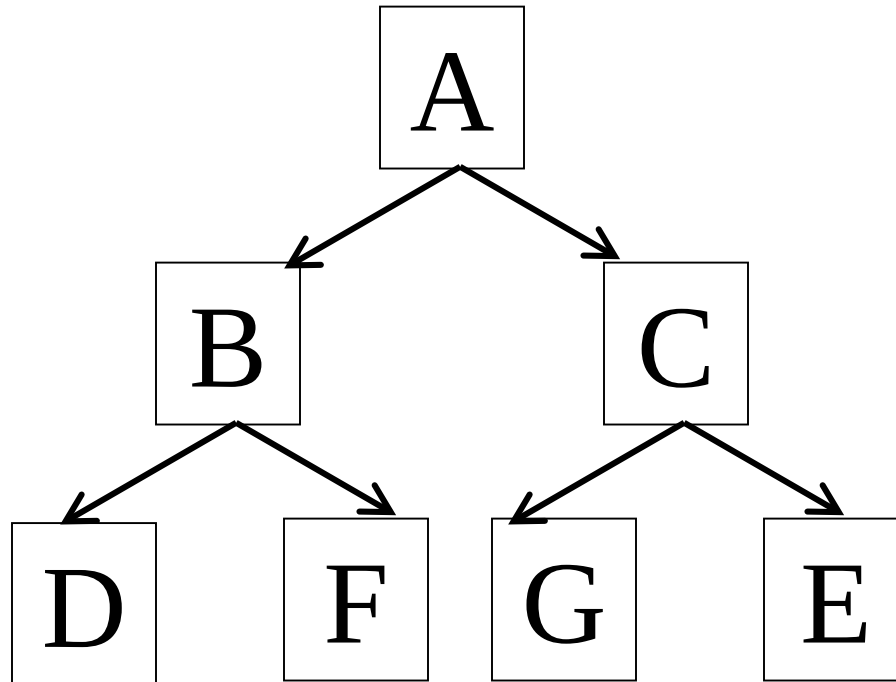
Kiểu dữ liệu tuyến tính

- Là kiểu dữ liệu thoả 02 tính chất sau:
 1. Đứng sau mỗi phần tử tối đa có 01 phần tử
 2. Mỗi phần tử có không quá 01 phần tử đứng trước nó.
- Ví dụ:

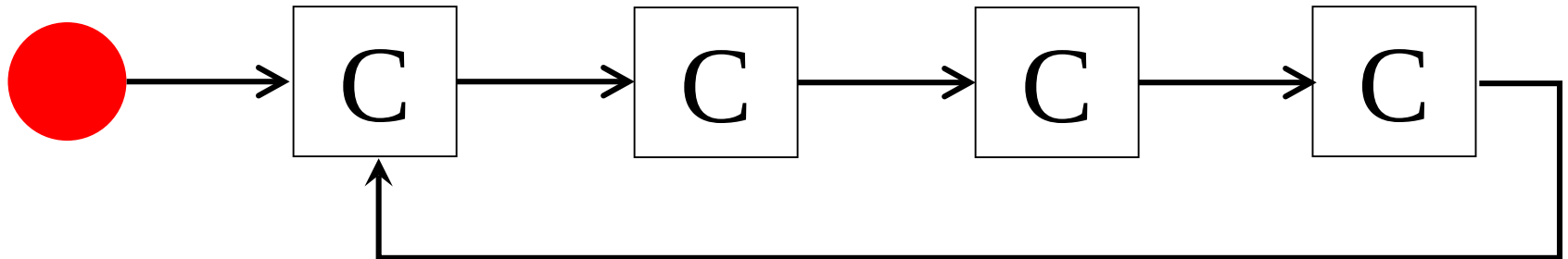
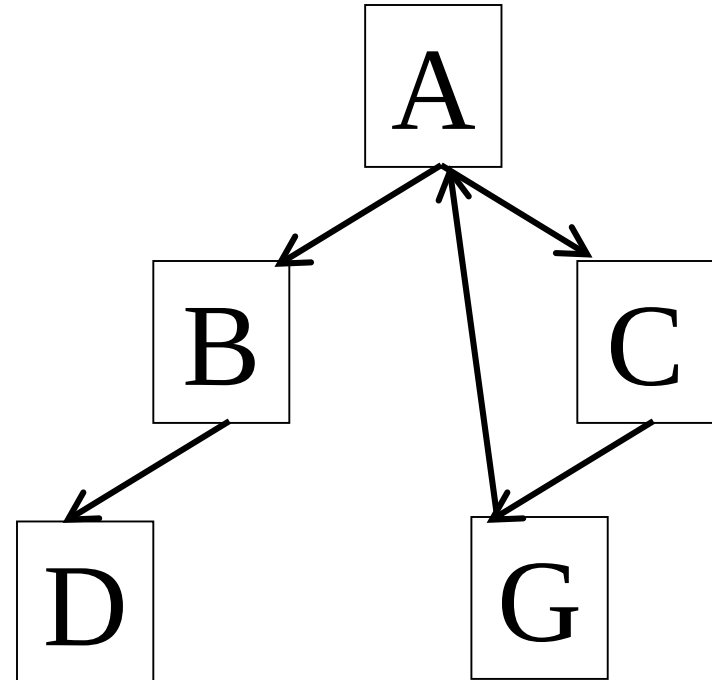
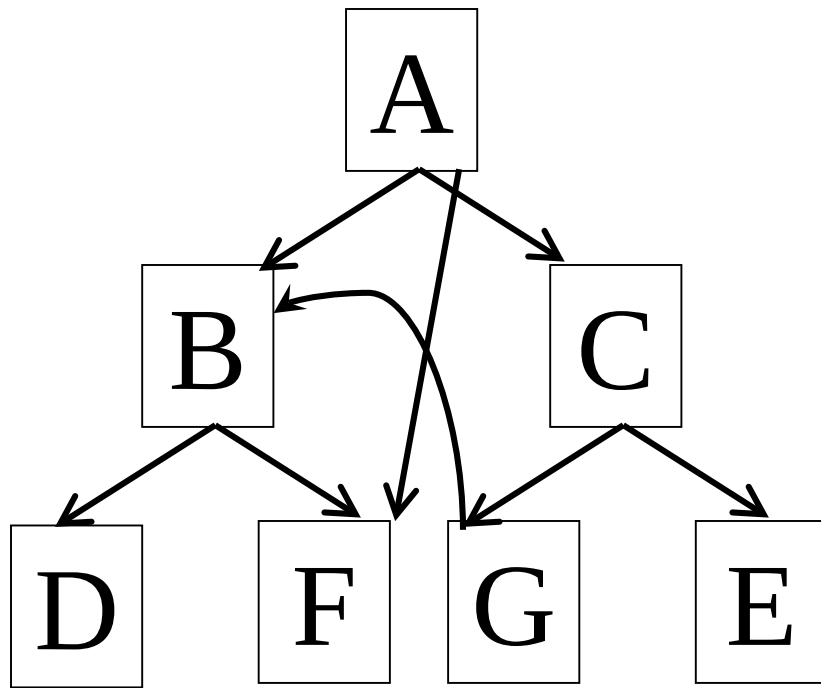


Kiểu dữ liệu phi tuyến

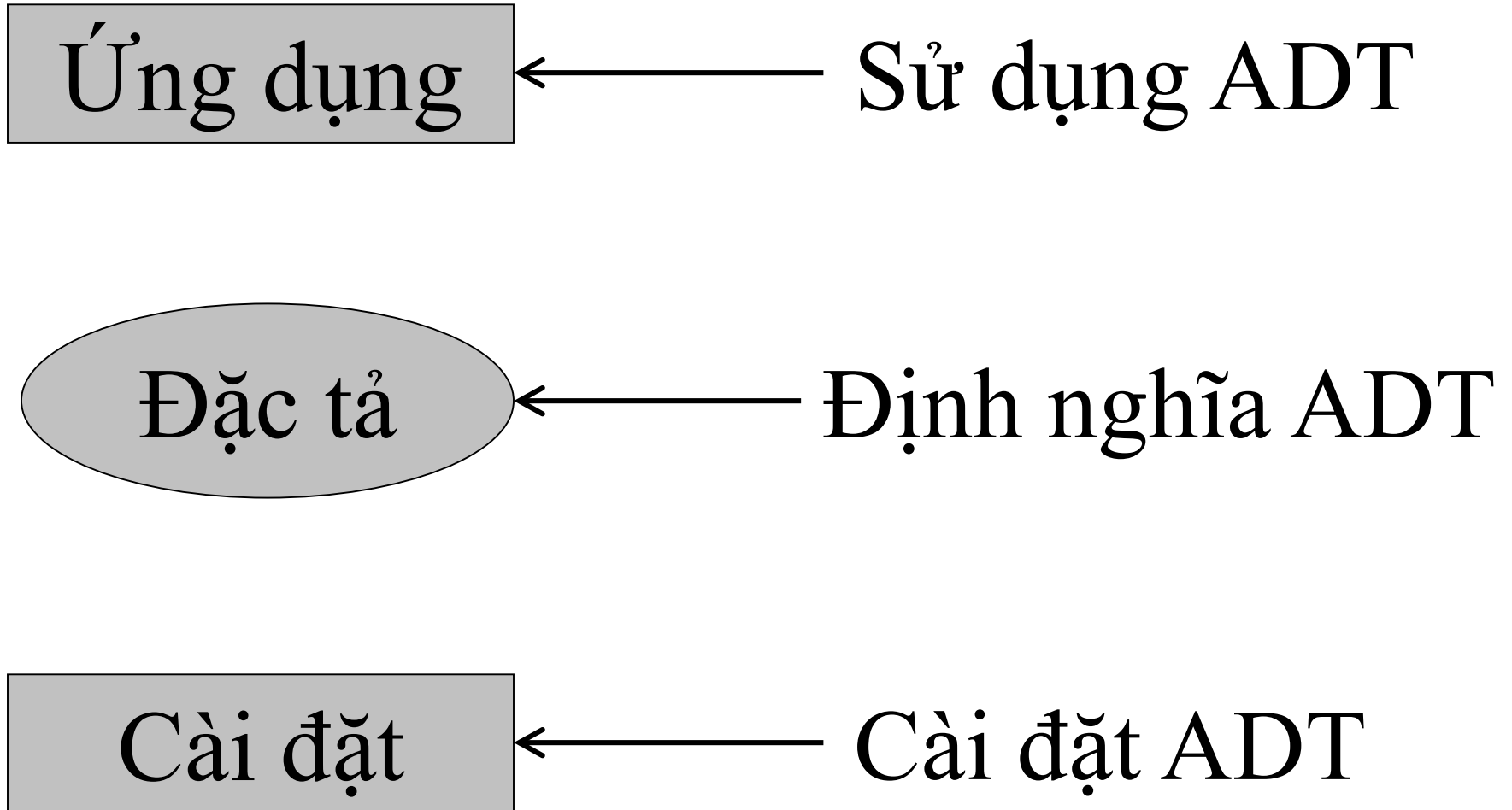
- Là KDL không thoả ít nhất 01 trong 02 tính chất của KDL tuyến tính trên
- Ví dụ:



Một số ví dụ



Sơ đồ xây dựng ADT



Ví dụ xây dựng một ADT đơn giản (KDL số phức)

- Đặc tả bài toán:

Số phức là một số biểu diễn bởi 01 cặp số nguyên (a,b) , a là phần thực và b là phần ảo. Ta có thể áp dụng các phép toán số học như: $+$, $-$, $*$, $/$. Ngoài ra ta có tính chất $i*i = -1$

- Cài đặt 1:

```
typedef struct {  
    int imaginary_number, real_number;  
}complex;
```


Ví dụ (tt)

- Cài đặt 2:

```
#define imaginary_number 0
```

```
#define real_number 1
```

```
typedef int complex[2];
```

- Các hàm cơ sở:

- ✓ createComplex(a,b): nhận 02 số nguyên a,b và trả ra số phức tương ứng.

- ✓ getImaginaryNumber(c): nhận vào 01 số phức c và trả ra phần ảo của nó.

- ✓ getRealNumber(c): nhận vào 01 số phức c và trả ra phần thực của nó.

Ví dụ (tt)

- Hàm mở rộng:

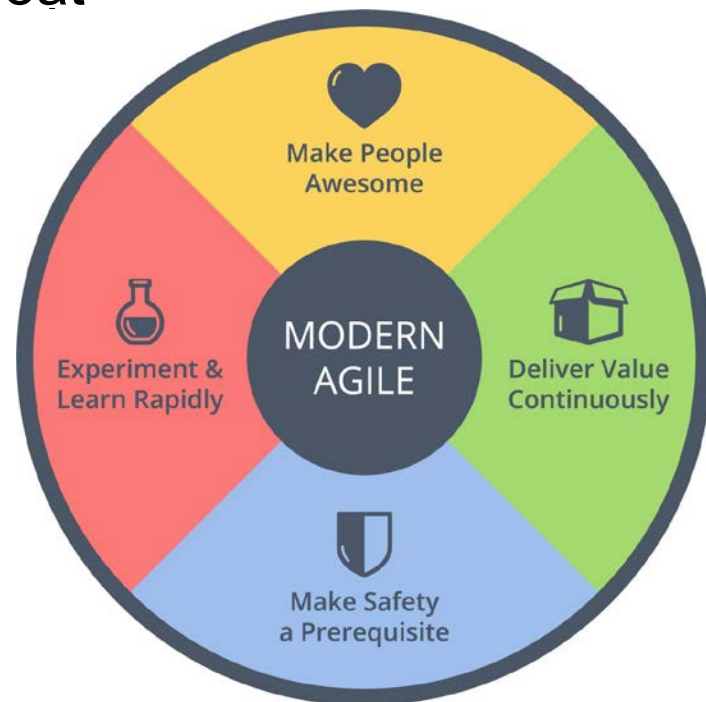
- ✓ `addComplex(complex c1, complex c2)`: cộng 02 số phức. Cài đặt như sau:

```
complex addComplex(complex c1, complex c2){  
    int r1 = getRealNumber(c1);  
    int r2 = getRealNumber(c2);  
    int i1 = getImaginaryNumber(c1);  
    int i2 = getImaginaryNumber(c2);  
    return createComplex(r1+r2,i1+i2);  
}
```

Agile

- Agile là một phương pháp phát triển phần mềm linh hoạt để làm sao đưa sản phẩm đến tay người dùng càng nhanh càng tốt càng sớm càng tốt.
- “Tuyên ngôn Phát triển phần mềm linh hoạt”

Cá nhân và sự tương tác hơn là
quy trình và công cụ;
Phần mềm chạy tốt hơn là tài liệu
đầy đủ;
Cộng tác với khách hàng hơn là
đàm phán hợp đồng;
Phản hồi với các thay đổi hơn là
bám sát kế hoạch.



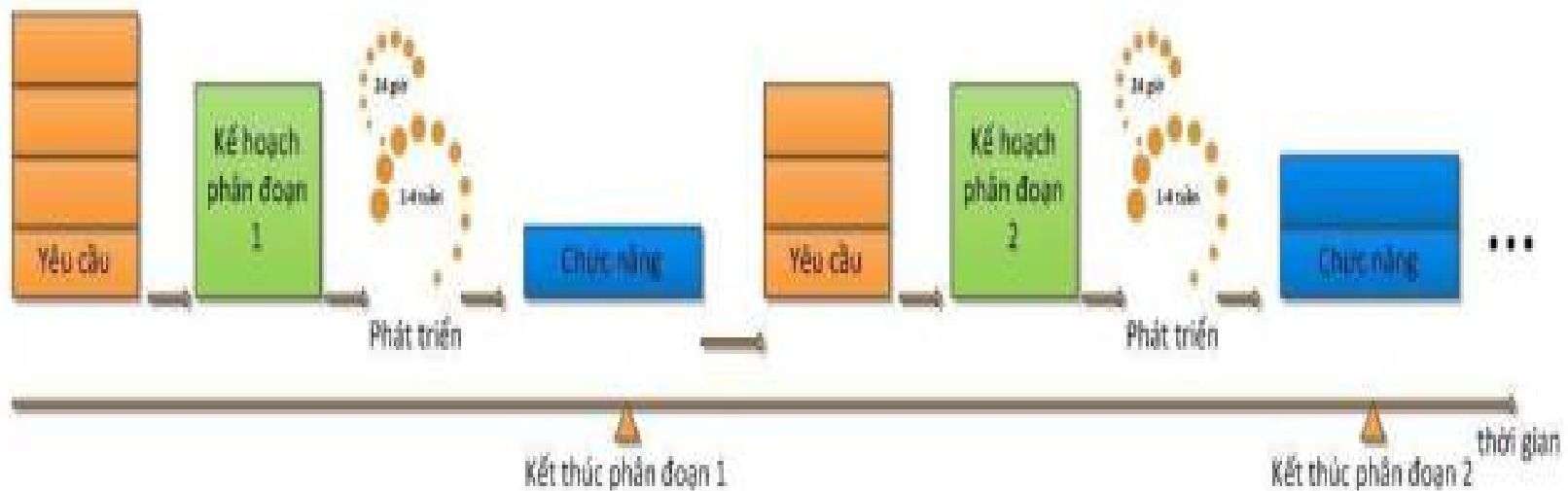
Các nguyên lý

1. Ưu tiên cao nhất của chúng tôi là thỏa mãn khách hàng thông qua việc chuyển giao sớm và liên tục các phần mềm có giá trị.
2. Chào đón việc thay đổi yêu cầu, thậm chí rất muộn trong quá trình phát triển. Các quy trình linh hoạt tận dụng sự thay đổi cho các lợi thế cạnh tranh của khách hàng.
3. Thường xuyên chuyển giao phần mềm chạy tốt tới khách hàng, từ vài tuần đến vài tháng, ưu tiên cho các khoảng thời gian ngắn hơn.
4. Nhà kinh doanh và nhà phát triển phải làm việc cùng nhau hàng ngày trong suốt dự án.
5. Xây dựng các dự án xung quanh những cá nhân có động lực. Cung cấp cho họ môi trường và sự hỗ trợ cần thiết, và tin tưởng họ để hoàn thành công việc.
6. Phương pháp hiệu quả nhất để truyền đạt thông tin tới nhóm phát triển và trong nội bộ nhóm phát triển là hội thoại trực tiếp.
7. Phần mềm chạy tốt là thước đo chính của tiến độ.
8. Các quy trình linh hoạt thúc đẩy phát triển bền vững. Các nhà tài trợ, nhà phát triển, và người dùng có thể duy trì một nhịp độ liên tục không giới hạn.
9. Liên tục quan tâm đến các kĩ thuật và thiết kế tốt để gia tăng sự linh hoạt.
10. Sự đơn giản – nghệ thuật tối đa hóa lượng công việc chưa xong – là căn bản.
11. Các kiến trúc tốt nhất, yêu cầu tốt nhất, và thiết kế tốt nhất sẽ được làm ra bởi các nhóm tự tổ chức.
12. Đội sản xuất sẽ thường xuyên suy nghĩ về việc làm sao để trở nên hiệu quả hơn, sau đó họ sẽ điều chỉnh và thay đổi các hành vi của mình cho phù hợp.

Đặc trưng Agile

1. Tính lặp (Iterative)

- Dự án sẽ được thực hiện trong các phân đoạn lặp đi lặp lại. Các phân đoạn (được gọi là Iteration hoặc Sprint) này thường có khung thời gian ngắn (từ 1 – 4 tuần).
- Trong mỗi phân đoạn này, nhóm phát triển thực hiện đầy đủ các công việc cần thiết như lập kế hoạch, phân tích yêu cầu, thiết kế, triển khai, kiểm thử (với các mức độ khác nhau) để cho ra các phần nhỏ của sản phẩm.
- Các phương pháp Agile thường phân rã mục tiêu thành các phần nhỏ với quá trình lập kế hoạch đơn giản và gọn nhẹ nhất có thể, và không thực hiện việc lập kế hoạch dài hạn.



2. Tính tiệm tiến (Incremental) và tiến hóa (Evolutionary)

- Cuối các phân đoạn, nhóm phát triển thường cho ra các phần nhỏ của sản phẩm cuối cùng.
- Các phần nhỏ này thường là đầy đủ, có khả năng chạy tốt, được kiểm thử cẩn thận và có thể sử dụng ngay (gọi là potentially shippable product increment of functionality).
- Theo thời gian, phân đoạn này tiếp nối phân đoạn kia, các phần chạy được này sẽ được tích lũy, lớn dần lên cho tới khi toàn bộ yêu cầu của khách hàng được thỏa mãn.

3. Tính thích ứng (hay thích nghi – adaptive)

- Do các phân đoạn chỉ kéo dài trong một khoảng thời gian ngắn, và việc lập kế hoạch cũng được điều chỉnh liên tục, nên các thay đổi trong quá trình phát triển (yêu cầu thay đổi, thay đổi công nghệ, thay đổi định hướng về mục tiêu v.v.) đều có thể được đáp ứng theo cách thích hợp.
- Theo đó, các quy trình Agile thường thích ứng rất tốt với các thay đổi.

4. Nhóm tự tổ chức và liên chức năng

- Cấu trúc nhóm Agile thường là liên chức năng (cross-functionality) và tự tổ chức (self-organizing).
- Theo đó, các nhóm này tự thực hiện lấy việc phân công công việc mà không dựa trên các mô tả cứng về chức danh (title) hay làm việc dựa trên một sự phân cấp rõ ràng trong tổ chức.
- Nhóm tự tổ chức có nghĩa là nó đã đủ các kĩ năng (competency) cần thiết cho việc phát triển phần mềm, do vậy nó có thể được trao quyền để tự ra quyết định, tự quản lí và tổ chức lấy công việc của chính mình để đạt được hiệu quả cao nhất.

5. Quản lý tiến trình thực nghiệm (Empirical Process Control)

- Các nhóm Agile ra các quyết định dựa trên các dữ liệu thực tiễn thay vì tính toán lý thuyết hay các tiền giả định (prescription).
- Nói cách khác, Agile rút ngắn vòng đời phản hồi (short feedback life cycle) để dễ dàng thích nghi và gia tăng tính linh hoạt.
- Theo thời gian, các chiến lược này sẽ tiến gần đến trạng thái tối ưu, nhờ đó nhóm có thể kiểm soát được tiến trình, và nâng cao năng suất lao động.

6. Giao tiếp trực diện (face-to-face communication)

- Về yêu cầu của khách hàng, Agile khuyến khích nhóm phát triển trực tiếp nói chuyện với khách hàng để hiểu rõ hơn về cái khách hàng thực sự cần, thay vì phụ thuộc nhiều vào các loại văn bản.
- Trong giao tiếp giữa nội bộ nhóm phát triển với nhau, thay vì một lập trình viên (thực hiện việc mã hóa) và một kỹ sư (thực hiện việc thiết kế) giao tiếp với nhau thông qua bản thiết kế.

7. Phát triển dựa trên giá trị (value-based development)

- Một trong các nguyên tắc cơ bản của Agile là “phần mềm chạy tốt chính là thước đo của tiến độ”. Nguyên tắc này giúp nhóm dám loại bỏ đi các công việc dư thừa không trực tiếp mang lại giá trị cho sản phẩm.
- Để vận hành được cơ chế “làm việc dựa trên giá trị”, nhóm Agile thường làm việc trực tiếp và thường xuyên với khách hàng (hay đại diện của khách hàng), cộng tác trực tiếp với họ để biết yêu cầu nào có độ ưu tiên cao hơn, mang lại giá trị hơn sớm nhất có thể cho dự án.
- Nhờ đó các dự án Agile thường giúp khách hàng tối ưu hóa được giá trị của dự án. Một cách gần như trực tiếp, Agile gia tăng đáng kể độ hài lòng của khách hàng.

Scrum

- Scrum là là một quy trình phát triển phần mềm theo phương pháp Agile. Vì thế, nó tuân thủ các nguyên tắc của Agile Manifesto.